Universidad Nacional Autónoma de México

Facultad de Ingeniería

Ejemplo Flyweight: Blackjack

Programación Orientada a Objetos

Profesor: Edgar Tista García

Integrantes

Robles Cancino Arturo Andres
Rodríguez Cárcamo Ditter Emmanuel
Santamaría Viveros César Yurem

2 de Noviembre, 2026

1 Diseño del ejemplo: blackjack con ocho mazos

Se modeló un juego de blackjack con un `Repartidor` de ocho mazos (416 cartas en juego). La baraja completa contiene únicamente 52 combinaciones distintas de $(palo, rango)$, lo cual sugiere de inmediato la aplicación del patrón. La Tabla 1 mapea los participantes con las clases del ejemplo.

Participante [1, p. 199]	**Clase en el ejemplo**
`ConcreteFlyweight`	`Carta`
`FlyweightFactory`	`CartaFactory`
`Client`	`Repartidor`, `Mano`
`UnsharedConcreteFlyweight`	(no se requiere)
`Flyweight` (interfaz)	(no se requiere: un solo concrete)

Table 1: Mapeo entre los participantes y las clases del ejemplo.

El proyecto se organiza en ocho archivos:

- `Palo.java`, `Rango.java`: enumeraciones que modelan el estado intrínseco primitivo. Por ser `enum`, la JVM garantiza una única instancia de cada constante.
- `CartaImagen.java`: simula el peso gráfico de un naipe (20 KB). Es el estado intrínseco pesado.
- `Carta.java`: el `ConcreteFlyweight`. Inmutable, con constructor de visibilidad de paquete.
- `CartaFactory.java`: la `FlyweightFactory`. Mantiene un caché `EnumMap` con las 52 cartas.
- `Repartidor.java`: el cliente que mantiene el orden extrínseco de las cartas.
- `Mano.java`: el cliente que mantiene el estado extrínseco por jugador.
- `DemoFlyweight.java`: la demostración y el análisis de memoria.

2 Cómo funciona el código

2.1 El flyweight: `Carta`

La clase `Carta` encapsula únicamente estado intrínseco: palo, rango e imagen. Es `final`, y sus campos igual, además de no poder instanciarse ya que debe ser inmutable por definición. Además, se dice que teniendo estas bases podemos generar un objeto `ConcreteFlyweight` el cual debería poder compartirse, pero ahora con datos particulares que los hacen intrínsecos [1, p. 199].

```
public final class Carta {
    private final Palo palo;
    private final Rango rango;
    private final CartaImagen imagen;

    Carta(Palo palo, Rango rango) {
        this.palo   = palo;
        this.rango  = rango;
        this.imagen = new CartaImagen();
    }
    // getters, equals, hashCode, describe...
}
```

Además, `Carta` expone `describe(boolean bocaArriba)`, que recibe estado extrínseco por parámetro y lo combina con su estado intrínseco para producir un resultado. Ese método materializa la colaboración descrita

en el libro donde dice que, el cliente pasa el estado extrínseco al flyweight cuando invoca sus operaciones [1, p. 199].

2.2 La fábrica: `CartaFactory`

La fábrica construye las 52 cartas una sola vez, en un método privado que se invoca al inicializar el campo estático `CACHE`. Usa `EnumMap` anidado, que indexa por el `ordinal()` del enum y ofrece acceso O(1) sin hashing.

```
private static final Map<Palo, Map<Rango, Carta>> CACHE =construirCache();

private static Map<Palo, Map<Rango, Carta>> construirCache() {
    Map<Palo, Map<Rango, Carta>> cache = new EnumMap<>(Palo.class);
    for (Palo palo : Palo.values()) {
        Map<Rango, Carta> porRango = new EnumMap<>(Rango.class);
        for (Rango rango : Rango.values()) {
            porRango.put(rango, new Carta(palo, rango));
        }
        cache.put(palo, Collections.unmodifiableMap(porRango));
    }
    return Collections.unmodifiableMap(cache);
}

public static Carta obtener(Palo palo, Rango rango) {
    return CACHE.get(palo).get(rango);
}
```

El método `obtener` no crea objetos: solo lee del caché. Todo el programa obtiene cartas a través de esta fábrica, cumpliendo la restricción fundamental del diseño, que es que los clientes no deben instanciar `ConcreteFlyweight` directamente, deben obtenerlos exclusivamente de la fábrica [1, p. 199]. En este ejemplo la restricción no se deja a la disciplina del programador: se impone a nivel del compilador con el constructor `package-private`.

2.3 Los clientes: `Repartidor` y `Mano`

`Repartidor` construye la lista de cartas llamando a la fábrica en bucles anidados. Cada elemento de la lista es una referencia a una de las 52 cartas, no una copia. El orden resultante es estado extrínseco y vive en la lista del repartidor.

```
public Repartidor(int numMazos) {
    List<Carta> temp = new ArrayList<>(numMazos * CARTAS_POR_MAZO);
    for (int m = 0; m < numMazos; m++) {
        for (Palo palo : Palo.values()) {
            for (Rango rango : Rango.values()) {
                temp.add(CartaFactory.obtener(palo, rango));
            }
        }
    }
    Collections.shuffle(temp);
    this.cartas = temp;
}
```

`Mano` modela el estado extrínseco por jugador mediante la clase anidada inmutable `CartaEnMano`, que asocia una carta (referencia al flyweight) con un booleano `bocaArriba`. Esta separación evita la trampa de identidad que discute [1, p. 197]: dos usos de la misma carta son lógicamente diferentes aunque apunten al mismo objeto.

2.4 Flujo completo

El flujo de una ejecución típica es el siguiente:

1. La JVM carga `CartaFactory` por primera vez. Se ejecuta `construirCache()` y se crean las 52 cartas. A partir de este momento, esas 52 instancias existen para siempre.

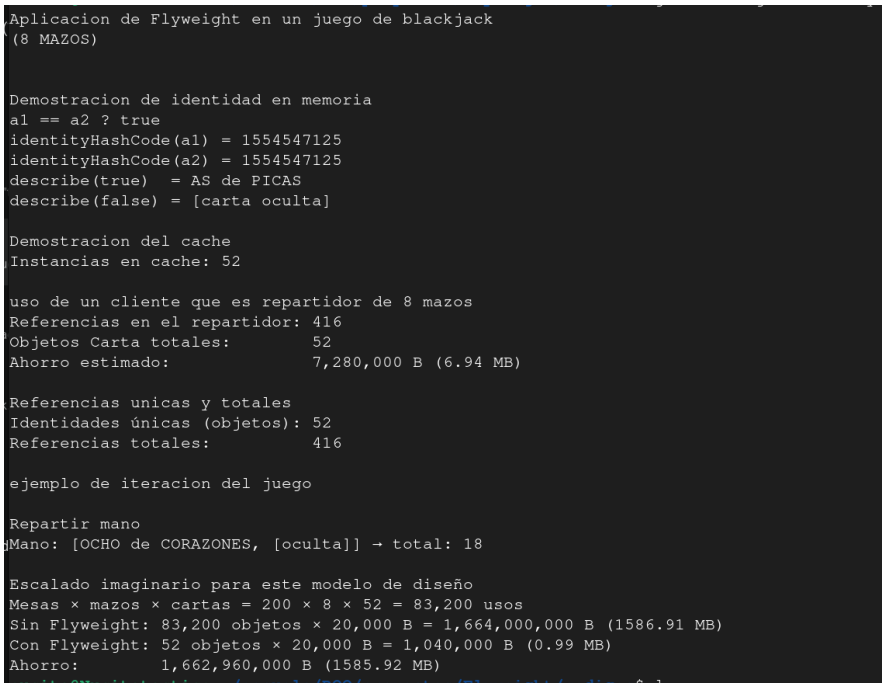
2. `DemoFlyweight.main` crea un `Repartidor` con 8 mazos. El constructor llama `CartaFactory.obtener` 416 veces; cada llamada devuelve una referencia, no un objeto nuevo.

3. El repartidor mezcla la lista y queda listo para repartir. La mezcla modifica el orden, no las cartas.

4. Al repartir una mano, el `Mano.recibir` envuelve la carta en `CartaEnMano`, agregando el estado extrínseco `bocaArriba`.

5. Al pedir una carta ya existente (por ejemplo, otra vez el As de picas), la fábrica devuelve la misma instancia. `System.identityHashCode` devuelve el mismo valor para ambas referencias.

Las siguientes imágenes son capturas de ejecución del código:



```
Aplicacion de Flyweight en un juego de blackjack
(8 MAZOS)


Demostracion de identidad en memoria
a1 == a2 ? true
identityHashCode(a1) = 1554547125
identityHashCode(a2) = 1554547125
describe(true)  = AS de PICAS
describe(false) = [carta oculta]

Demostracion del cache
Instancias en cache: 52

uso de un cliente que es repartidor de 8 mazos
Referencias en el repartidor: 416
Objetos Carta totales:        52
Ahorro estimado:              7,280,000 B (6.94 MB)

Referencias unicas y totales
Identidades únicas (objetos): 52
Referencias totales:          416

ejemplo de iteracion del juego

Repartir mano
Mano: [AS de PICAS, [oculta]] → total: 22

Escalado imaginario para este modelo de diseño
Mesas × mazos × cartas = 200 × 8 × 52 = 83,200 usos
Sin Flyweight: 83,200 objetos × 20,000 B = 1,664,000,000 B (1586.91 MB)
Con Flyweight: 52 objetos × 20,000 B = 1,040,000 B (0.99 MB)
Ahorro:        1,662,960,000 B (1585.92 MB)
```

3 Estado intrínseco y extrínseco

La Tabla 2 resume la separación.

Concepto	**En el ejemplo**	**Dónde vive**
Estado intrínseco	palo, rango, imagen	`Carta`
Estado extrínseco (orden)	posición en el `Repartidor`	`Repartidor`
Estado extrínseco (uso)	dueño, boca arriba	`Mano.CartaEnMano`

Table 2: Separación entre estado intrínseco y extrínseco.

4 ¿Por qué es adecuado aplicar Flyweight aquí?

El libro enumera cinco condiciones que deben cumplirse para que el patrón sea aplicable [1, p. 197]. A continuación se verifica cada una en el ejemplo.

1. **La aplicación usa una gran cantidad de objetos.** La demo escala los cálculos a 200 mesas × 8 mazos × 52 cartas = 83,200 usos. Aunque el ejemplo concreto use ocho mazos, el modelo escala linealmente con el número de mesas.

2. **Los costos de almacenamiento son altos por la cantidad de objetos.** Si cada `Carta` cargara su propia `CartaImagen` de 20 KB, 83,200 objetos ocuparían aproximadamente 1.6 GB. Con Flyweight, las 52 imágenes compartidas ocupan apenas 1 MB.

3. **La mayor parte del estado puede hacerse extrínseco.** Palo, rango e imagen son independientes del contexto; posición, dueño y visibilidad se calculan o almacenan en el cliente.

4. **Grupos de objetos pueden reemplazarse por unos pocos compartidos.** Las 83,200 instancias potenciales se reemplazan por 52 objetos compartidos más 83,200 referencias.

5. **La aplicación no depende de la identidad de los objetos.** El código evita comparar cartas con `==` para distinguir usos; `CartaEnMano` modela el «cuál» y `Carta` modela el «qué». Esto es algo que se

menciona en el libro como un criterio importante; así se distingue cuáles objetos entran en juego en cada bloque de código.

4.1 Análisis de memoria

La demo imprime el uso real de memoria al construir el repartidor y al solicitar cartas existentes. Los valores típicos son:

- Construir `Repartidor(8)`: aproximadamente 8 KB asignados (416 referencias más el overhead de la lista).
- Solicitar una `Carta` ya existente: 0 bytes asignados, porque la fábrica devuelve una referencia sin crear objeto.

Este último valor es el argumento central del patrón: pedir un flyweight existente no genera nuevas instancias.

Conclusión

Se considera que este ejemplo puede englobar un pequeño ejemplo escalable de cómo se puede llegar a aplicar el modelo Flyweight para diseñar programas donde se repiten grandes cantidades de datos que, gracias a un análisis detallado, se puede llegar a clasificar las características del sistema en intrínsecos y extrínsecos, para así dar una solución más amigable con la memoria.

Referencias

[1] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software* (Addison-Wesley Professional Computing Series). Reading, MA: Addison-Wesley, 1994, ISBN: 0-201-63361-2.